

Tarea1.SimulacionEstocastica

Pablo León Evia

2026-08-30

Problema de la aguja de Buffon

Contexto: El experimento mental consiste en dejar caer una aguja de una longitud determinada sobre una superficie plana que tiene dibujadas líneas paralelas equidistantes, separadas a una distancia fija. La condición principal del problema establece que la longitud de la aguja debe ser menor o igual a la distancia que separa a las líneas. El objetivo es determinar la probabilidad exacta de que la aguja caiga de tal forma que cruce o toque al menos una de las líneas del suelo.

Se definirá el experimento de Buffon como una función en el lenguaje R, para facilidad de llamar la función con parámetros arbitrarios.

```
l <- 1 # Longitud de la aguja
w <- 2 # Ancho de las lineas
n <- 100 # Número de agujas

buffon_simulacion <- function(L, W, N) {

  set.seed(12131)

  theta <- runif(N, 0, pi)
  x <- runif(N, 0, 10)      # Distribución horizontal para la gráfica
  y <- runif(N, 0, 4*W)    # Se puso 4 para que no se acumulen todos en una franja nada más

  # Calcula si cruza o no, se tienen que cumplir una de las dos restricciones (| = 0)
  cruces <- (y + L/2 * abs(sin(theta)) >= W * floor(y/W) + W) |
            (y - L/2 * abs(sin(theta)) <= W * floor(y/W))
  # Conteo de las que cruzaron
  M <- sum(cruces)

  # Fórmula despejada
  pi_estimado <- 2 * N / (M * W)

  # Grafico, creamos una tabla (dataframe) para agrupar los datos de cada aguja
  # (Inicio, Fin y Angulo)
  df_agujas <- data.frame(
    x1 = x - (L / 2) * cos(theta),
    x2 = x + (L / 2) * cos(theta),
    y1 = y - (L / 2) * sin(theta),
    y2 = y + (L / 2) * sin(theta),
    Estado = ifelse(cruces, "Cruzó", "No cruzó")
  )
}
```

```

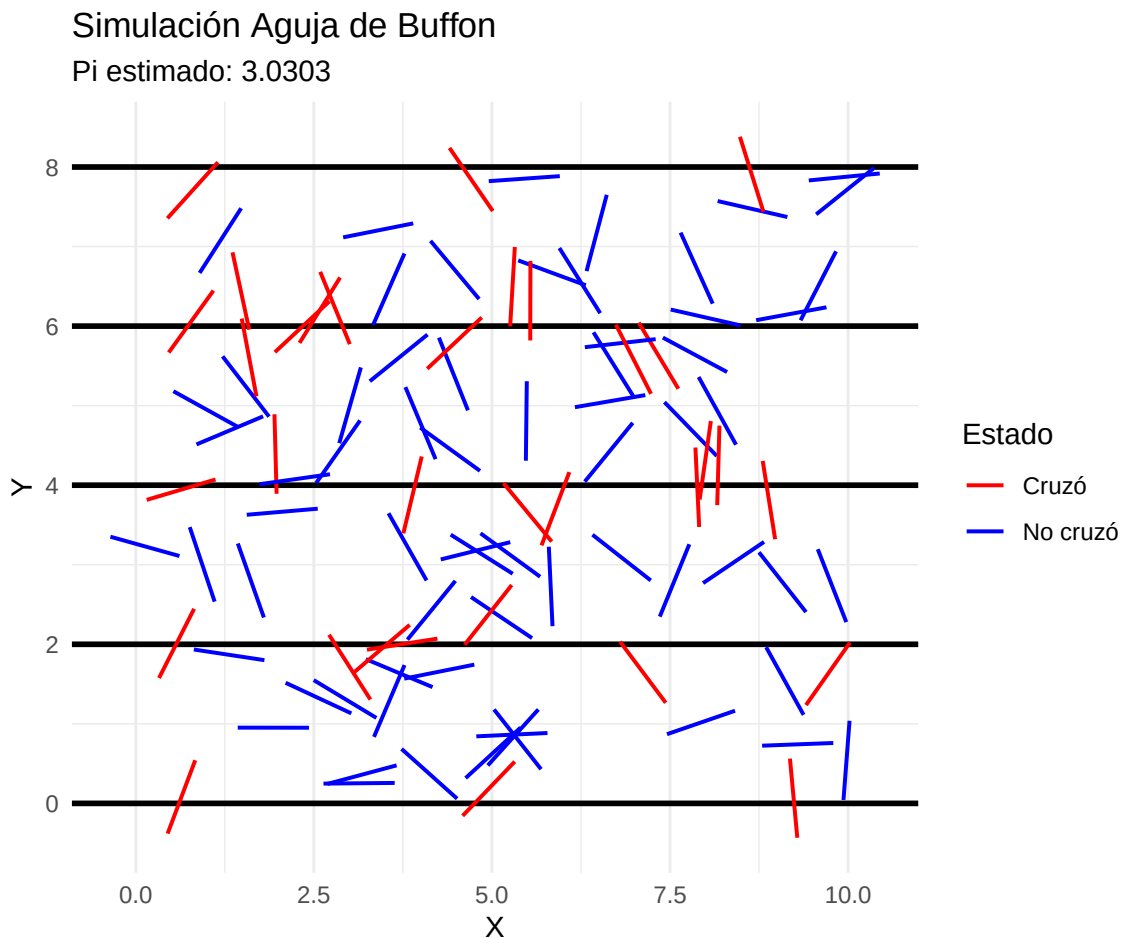
p <- ggplot() +
  geom_hline(yintercept = seq(0, 4 * W, by = W), color = "black", linewidth = 1) +
  geom_segment(data = df_agujas,
    aes(x = x1, y = y1, xend = x2, yend = y2, color = Estado),
    linewidth = 0.7) +
  scale_color_manual(values = c("Cruzó" = "red", "No cruzó" = "blue")) +
  labs(
    title = "Simulación Aguja de Buffon",
    subtitle = paste("Pi estimado:", round(pi_estimado, 4)),
    x = "X", y = "Y"
  ) +
  theme_minimal()

print(p)

return(pi_estimado)
}

# La ejecutas pasando únicamente L, W y N:
resultado <- buffon_simulacion(L = l, W = w, N = n)

```



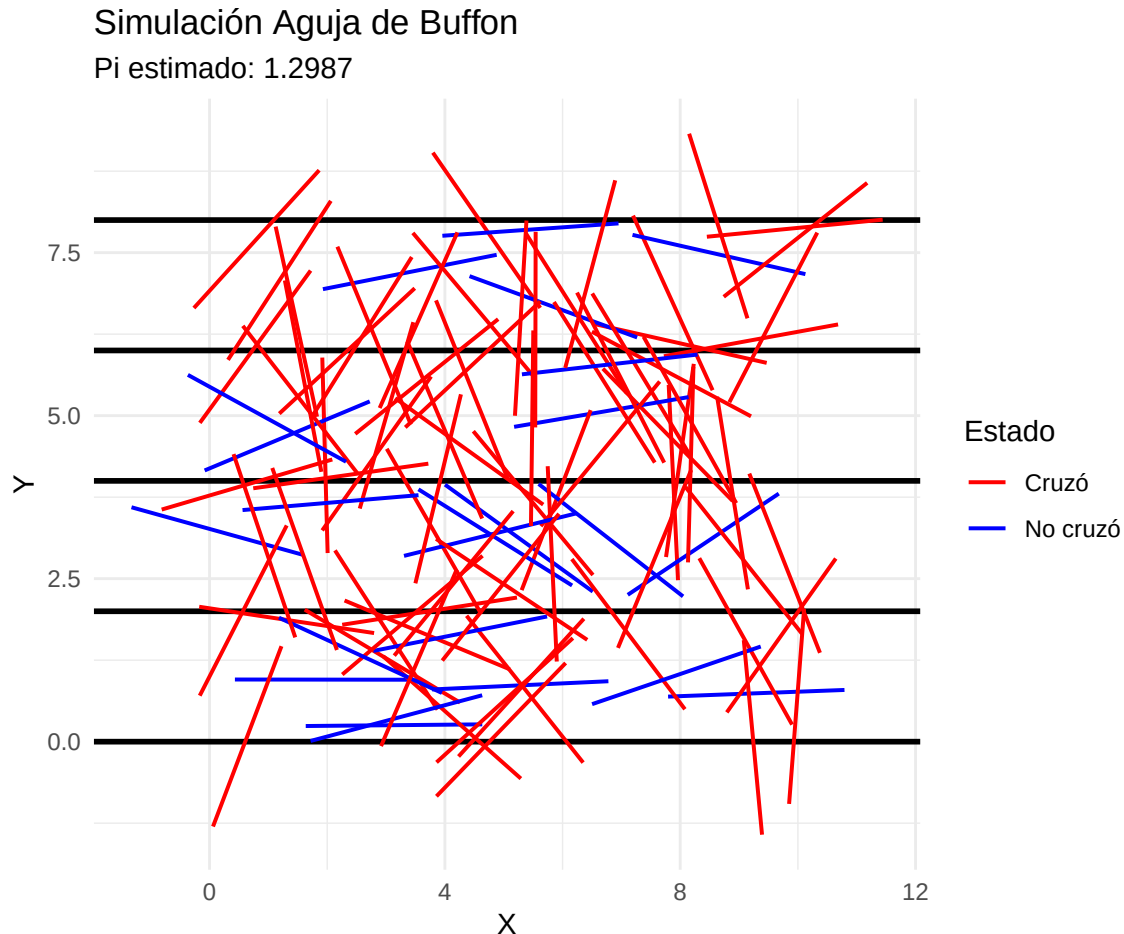
Estableciendo una semilla igual a 12131 se tiene una estimación de π de 3.0303 para 100 agujas, se creó una

función que es flexible al número de parámetros requeridos.

b) Considera que la aguja tiene mayor tamaño que la separación entre líneas, esto es, $l > t$. ¿El código visto en clase sigue funcionando de manera adecuada??

Llamamos a la función `buffon_simulacion` con los parámetros que requerimos, en este caso, el texto se presta a interpretación, solo que se cumpla que la aguja es mayor al número t que en el contexto de nuestro código es W (La anchura entre cada línea)

```
buffon_simulacion(L=1 + 2, W=w, N=n)
```



```
## [1] 1.298701
```

Es claro observar que la estimación es muy poco precisa en los casos que la longitud de la aguja es mayor que la anchura que hay entre los segmentos de líneas paralelas, el código no funciona como se desea.

c) Obtén o consulta una expresión para la probabilidad de cruce cuando $l > t$, justifica la expresión utilizada y modifica el código para que funcione en ambos casos

En el libro Geometrical Probability by Kendall; en la página 70 se encuentra esta fórmula para el caso $L > t$:

$$P = \frac{2}{\pi} \left\{ \frac{\pi}{2} - \sin^{-1}(L^{-1}) + L - \sqrt{L^2 - 1} \right\}$$

Podemos hacer un simple despeje y obtenemos una estimación de π :

$$\pi = \frac{2L - 2 \sin^{-1}(L^{-1}) - 2\sqrt{L^2 - 1}}{\hat{P} - 1}$$

```
buffon_simulacion2 <- function(L, W, N) {

  set.seed(12131)

  theta <- runif(N, 0, pi)
  x <- runif(N, 0, 10)      # Distribución horizontal para la gráfica
  y <- runif(N, 0, 4*W)     # Se puso 4 para que no se acumulen todos en una franja nada más

  # Calcula si cruza o no, se tienen que cumplir una de las dos restricciones (| = 0)
  cruces <- (y + L/2 * abs(sin(theta)) >= W * floor(y/W) + W) |
            (y - L/2 * abs(sin(theta)) <= W * floor(y/W))
  # Conteo de las que cruzaron
  M <- sum(cruces)

  # Fórmula despejada, en teoría la fórmula se creó para el caso W = 1, lo que
  # hago es la proporción, por eso mismo divido L/W, para no reestringirme al caso w=1
  pi_estimado <- (+2 * (L/W) - 2 * asin(W/L) - 2 * sqrt((L/W)^2-1))/(M/N-1)

  # Grafico, creamos una tabla (dataframe) para agrupar los datos de cada aguja #
  # (Inicio, Fin y Angulo)
  df_agujas <- data.frame(
    x1 = x - (L / 2) * cos(theta),
    x2 = x + (L / 2) * cos(theta),
    y1 = y - (L / 2) * sin(theta),
    y2 = y + (L / 2) * sin(theta),
    Estado = ifelse(cruces, "Cruzó", "No cruzó")
  )

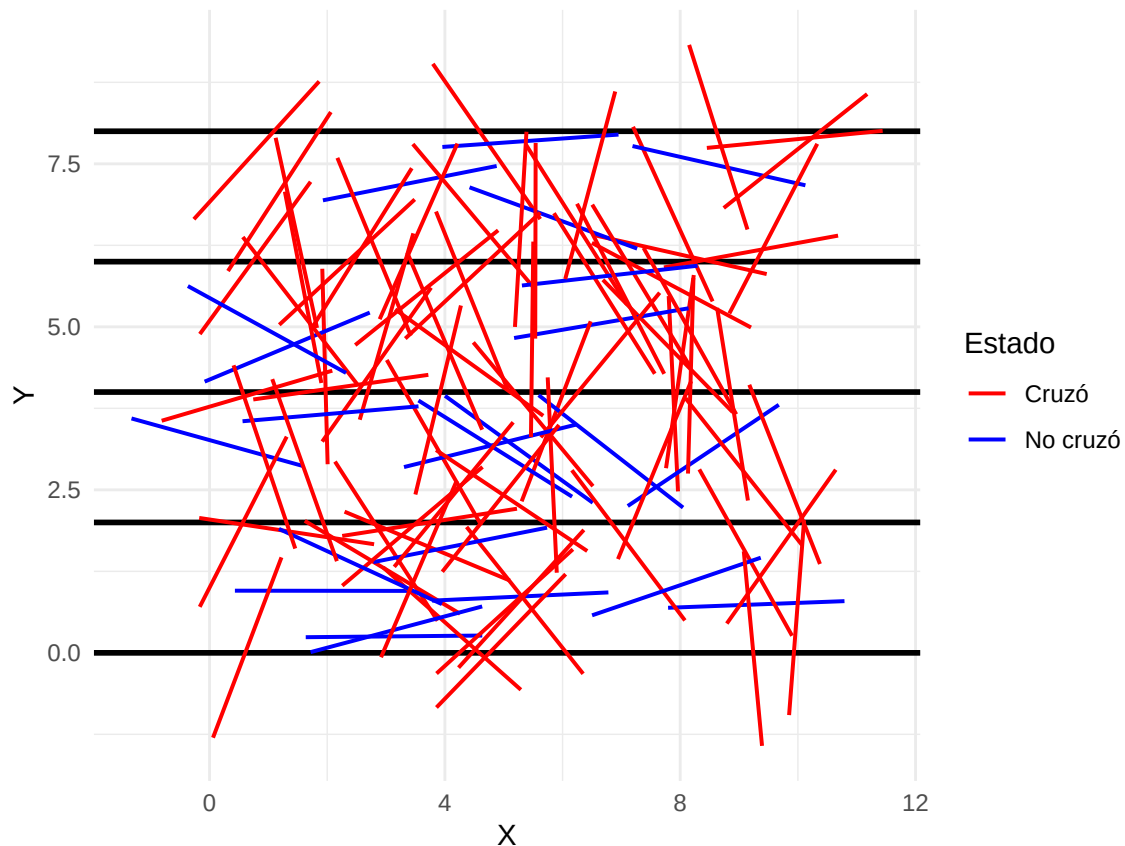
  p <- ggplot() +
    geom_hline(yintercept = seq(0, 4 * W, by = W), color = "black", linewidth = 1) +
    geom_segment(data = df_agujas,
      aes(x = x1, y = y1, xend = x2, yend = y2, color = Estado),
      linewidth = 0.7) +
    scale_color_manual(values = c("Cruzó" = "red", "No cruzó" = "blue")) +
    labs(
      title = "Simulación Aguja de Buffon",
      subtitle = paste("Pi estimado:", round(pi_estimado, 4)),
      x = "X", y = "Y"
    ) +
    theme_minimal()

  print(p)

  return(pi_estimado)
}
buffon_simulacion2(L=(1 + 2), W=w, N=n)
```

Simulación Aguja de Buffon

Pi estimado: 3.024



```
## [1] 3.024014
```

Estimar π a partir del círculo unitario:

2. Otra manera de estimar a π es considerando un círculo de radio 1 incrustado en un cuadrado como el que se ve en la Figura 1

```
ggplot() +
  # Cuadrado rojo de [-1, 1] en X y Y
  geom_rect(aes(xmin = -1, xmax = 1, ymin = -1, ymax = 1),
    fill = NA, color = "red", linewidth = 1.2) +
  # Círculo inscrito centrado en (0,0) con radio 1
  geom_circle(aes(x0 = 0, y0 = 0, r = 1),
    color = "black", linewidth = 0.8) +
  # Ejes punteados en x = 0 y y = 0
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray70") +
  geom_vline(xintercept = 0, linetype = "dashed", color = "gray70") +
  # Relación de aspecto 1:1 y límites
  coord_fixed(xlim = c(-1.1, 1.1), ylim = c(-1.1, 1.1)) +
  # Formato limpio igual al de la imagen
  theme_minimal() +
  theme(
```

```
axis.title = element_blank(),
panel.grid.minor = element_blank()
)
```

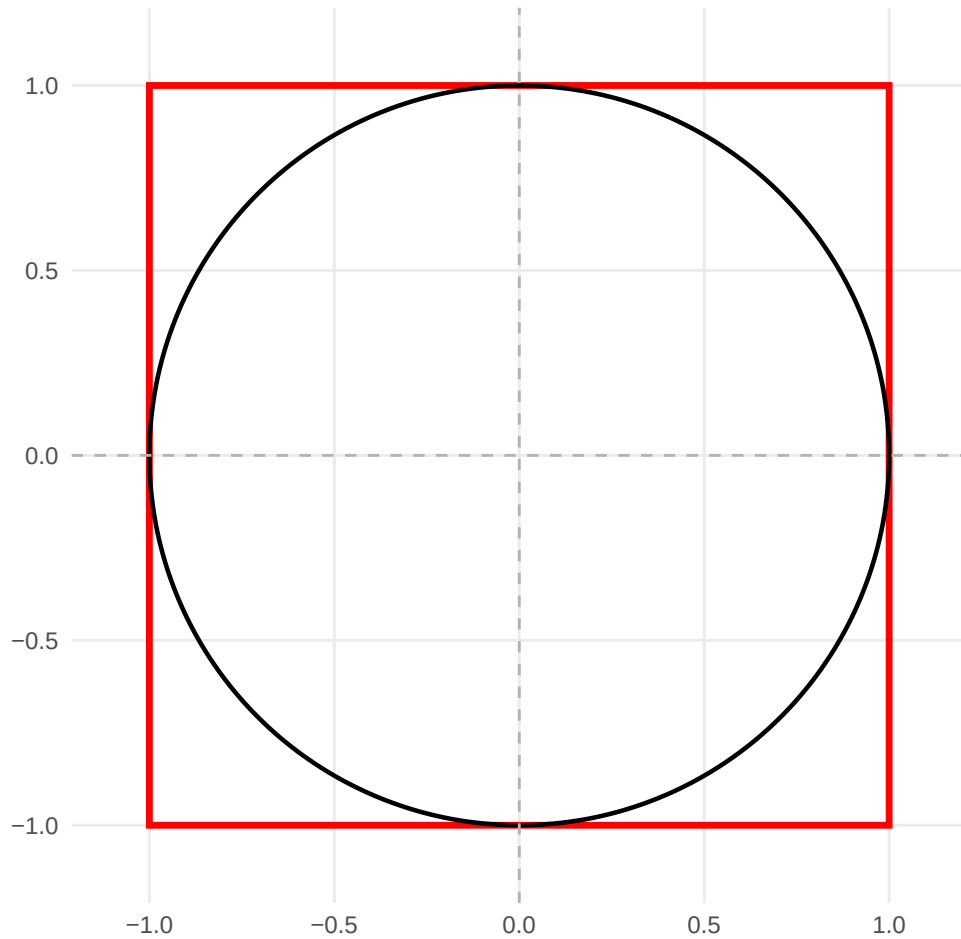


Figure 1: Figura 1: Círculo unitario inscrito en un cuadrado de lado 2.

El problema es el mismo si tomamos el cuadrado $[0,1] \times [0,1]$, debido a la simetría del área de la circunferencia y el del cuadrado; se generarán N puntos aleatorios, se proyectarán en el plano positivo y se tomará la proporción de puntos que cayeron dentro de la circunferencia entre el total de puntos generados, tenemos que para una cuarta parte de una circunferencia, el área es:

$$A = \frac{\pi r^2}{4}$$

Y el despeje, para estimar π es:

$$\pi = \frac{4 * \hat{A}}{r^2}$$

Es sencillo expresar la circunferencia en el plano, pero haremos énfasis a la parte positiva del plano, despejando queda:

$$y = \sqrt{1 - x^2} \quad \text{para } x \in [0, 1] \text{ y } y \in [0, 1]$$

Para saber si cae dentro de la circunferencia, solo se debe de cumplir la desigualdad:

$$0 \leq y \leq \sqrt{1 - x^2} \quad \text{con } 0 \leq x \leq 1$$

a) ¿Cuál es la probabilidad de que un punto lanzado al azar caiga dentro del círculo? Siguiendo el concepto de la regla de Laplace, es dividir el área de la circunferencia entre el área del cuadrado, para que sea continuo el problema, tomaré igual el área del plano positivo.

$$P = \frac{\frac{\pi}{4}}{1} = \frac{\pi}{4} \approx 0.7854$$

b) ¿Cómo puedes utilizar este resultado para aproximar a π ? Escribe un programa en el lenguaje de tu preferencia que te permita aproximar a π utilizando este método. Definí una función que recibe como parámetro el número de puntos usados, hace la estimación con las fórmulas de arriba y con la librería ggplot2 se genera un gráfico ilustrativo de los puntos.

```
pi_estimado <- function(N) {
  # Definimos los puntos generados, creamos la coordenada X y Y
  x <- runif(N,0,1)
  y <- runif(N,0,1)

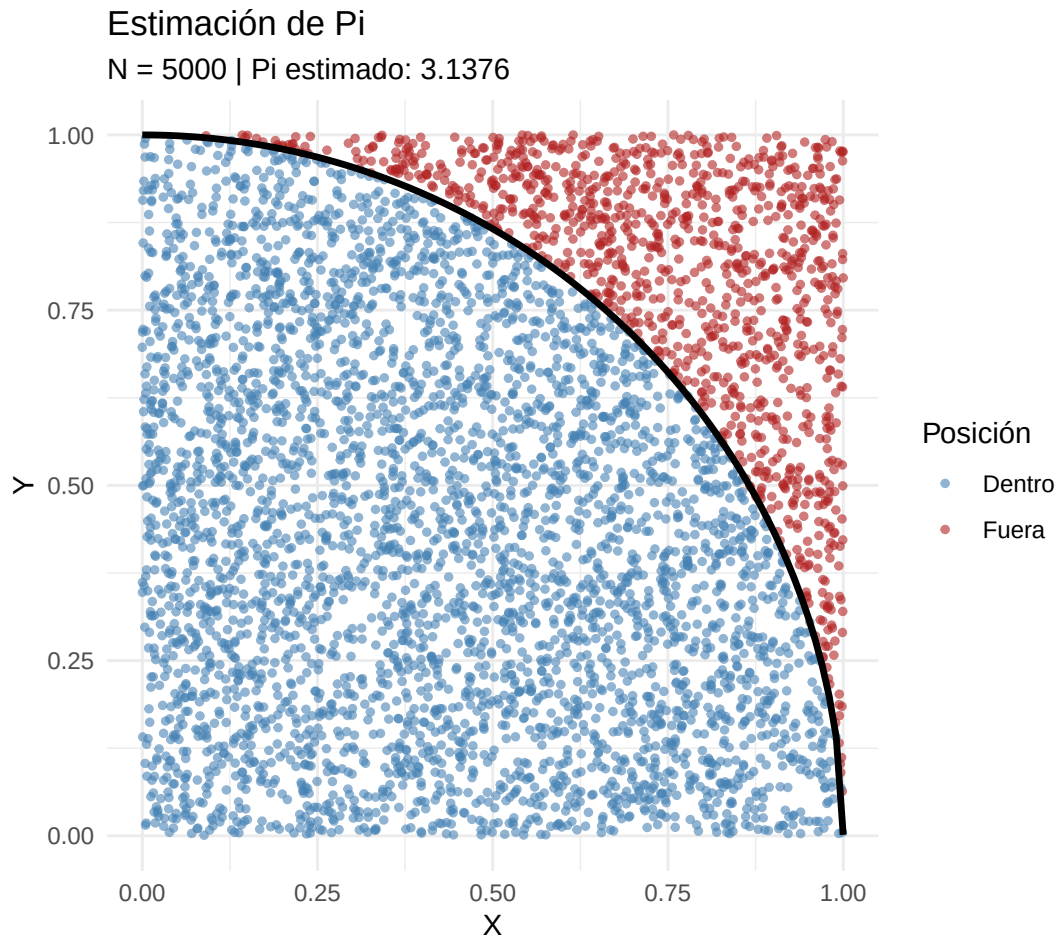
  # Calcula cuantos puntos cumplen la reestricción
  Incircunferencia <- (y) <= sqrt(1-x^2)
  Area_estimada = sum(Incircunferencia)/N
  pi_est = Area_estimada*4

  # 4. Crear DataFrame para graficar
  df_puntos <- data.frame(
    x = x,
    y = y,
    Estado = ifelse(Incircunferencia, "Dentro", "Fuera")
  )

  # 5. Construir gráfico en ggplot2
  p <- ggplot(df_puntos, aes(x = x, y = y)) +
    # Puntos simulados
    geom_point(aes(color = Estado), alpha = 0.6, size = 1) +
    # Curva del cuarto de círculo (y = sqrt(1 - x^2))
    stat_function(fun = function(x) sqrt(1 - x^2), color = "black", linewidth = 1.2) +
    # Colores personalizados
    scale_color_manual(values = c("Dentro" = "steelblue", "Fuera" = "firebrick")) +
    # Fijar límites del cuadrado [0, 1] x [0, 1] y relación de aspecto 1:1
    coord_fixed(xlim = c(0, 1), ylim = c(0, 1)) +
    labs(
      title = "Estimación de Pi",
      subtitle = paste("N =", N, "| Pi estimado:", round(pi_est, 4)),
      x = "X", y = "Y",
      color = "Posición"
    ) +
    theme_minimal()

  print(p)
  return(pi_est)
}

# --- LLAMADA A LA FUNCIÓN OBLIGATORIA PARA DESPLEGAR EL GRÁFICO ---
pi_estimado(5000)
```



```
## [1] 3.1376
```

c) Para $n = 100, 1000, 10000$ y 100000 , replica 100 veces cada experimento y calcula el error cuadrático medio (ECM) de las aproximaciones obtenidas para ambos métodos. Grafica el ECM en función de n y compara los resultados. ¿Cuál de los dos métodos parece ser más eficiente? Justifica tu respuesta.

Para esta pregunta, crearé una función que tendrá la misma idea que la que responde la pregunta b), para evitar generar n imágenes y crearé una función anidada que incluya la primera función de `pi_estimado` n veces.

```
# Quitamos la semilla para que no se reproduzca n veces el mismo resultado
set.seed(NULL)
```

```
pi_estimado <- function(N) {
  x <- runif(N,0,1)
  y <- runif(N,0,1)
  Incircunferencia <- (y <= sqrt(1-x^2))
  Area_estimada = sum(Incircunferencia)/N
  pi_est = Area_estimada*4
  return(pi_est)
}
```

```
# Funcion anidada
```

```

ecm <- function(N, n) {
  estm_Nx1 <- numeric(n)
  for (i in 1:n) {
    estm_Nx1[i] <- pi_estimado(N)
  }
  y_mean <- mean(estm_Nx1)

  ecm_in <- mean((estm_Nx1 - pi)^2)

  # --- GRÁFICA DENTRO DE LA FUNCIÓN ---
  df_sim <- data.frame(
    Simulacion = 1:n,
    Estimacion = estm_Nx1
  )

  p <- ggplot(df_sim, aes(x = Simulacion, y = Estimacion)) +
    geom_point(color = "steelblue", alpha = 0.7) +
    geom_hline(yintercept = pi, color = "firebrick", linetype = "dashed", linewidth = 1) +
    labs(
      title = paste("Simulaciones de Pi para N =", N, "y n =", n),
      subtitle = paste("ECM:", round(ecm_in, 6)),
      x = "Número de Simulación",
      y = "Valor estimado de Pi"
    ) +
    theme_minimal()

  print(p)
  cat('El valor de pi estimado es: ', y_mean, '\n')
  cat('El calculo del error cuadrático medio (ECM): ', ecm_in, '\n')

  return(ecm_in)
}

```

Para un $n = 100$, se tiene que la función definida arriba es:

```
## El valor de pi estimado es: 3.1188
## El calculo del error cuadrático medio (ECM): 0.02274207
```

```
## [1] 0.02274207
```

Para un $n = 1000$:

```
ecm(1000,100)
```

```
## El valor de pi estimado es: 3.13616
## El calculo del error cuadrático medio (ECM): 0.002256368
```

```
## [1] 0.002256368
```

Para un $n = 10000$:

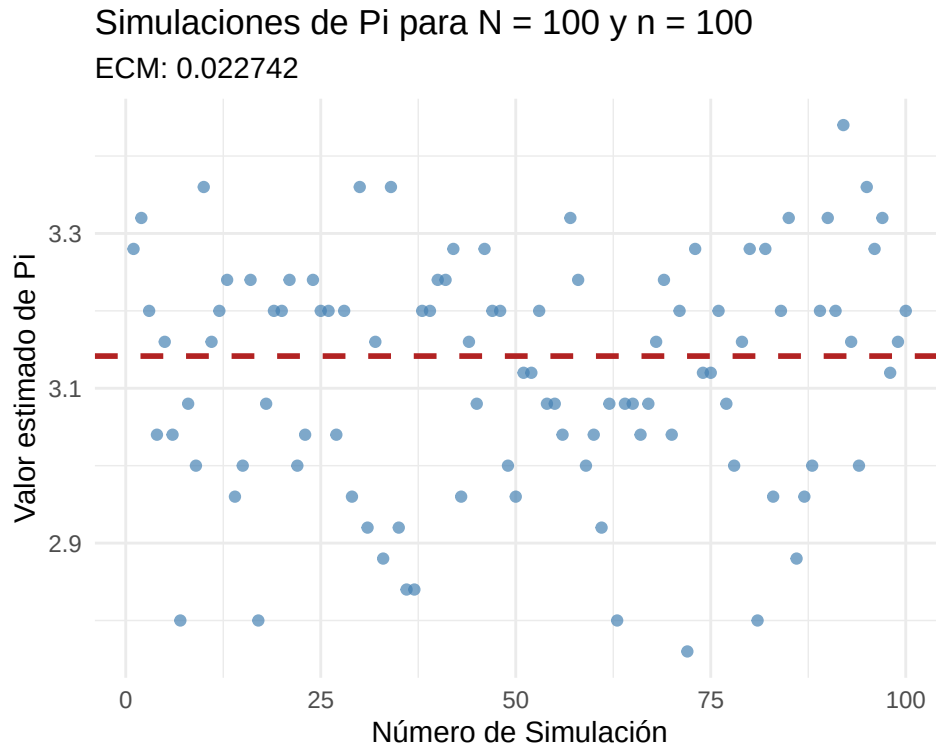


Figure 2: Figura 2: Simulación y cálculo del ECM con un tamaño de muestra de 100

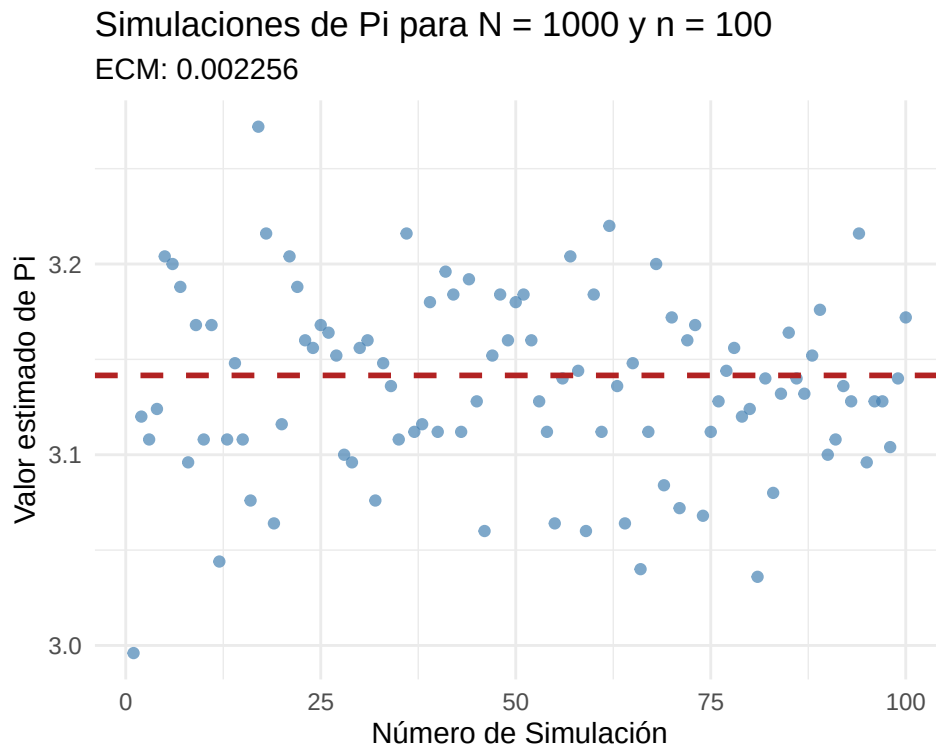


Figure 3: Figura 3: Simulación y cálculo del ECM con un tamaño de muestra de 1000

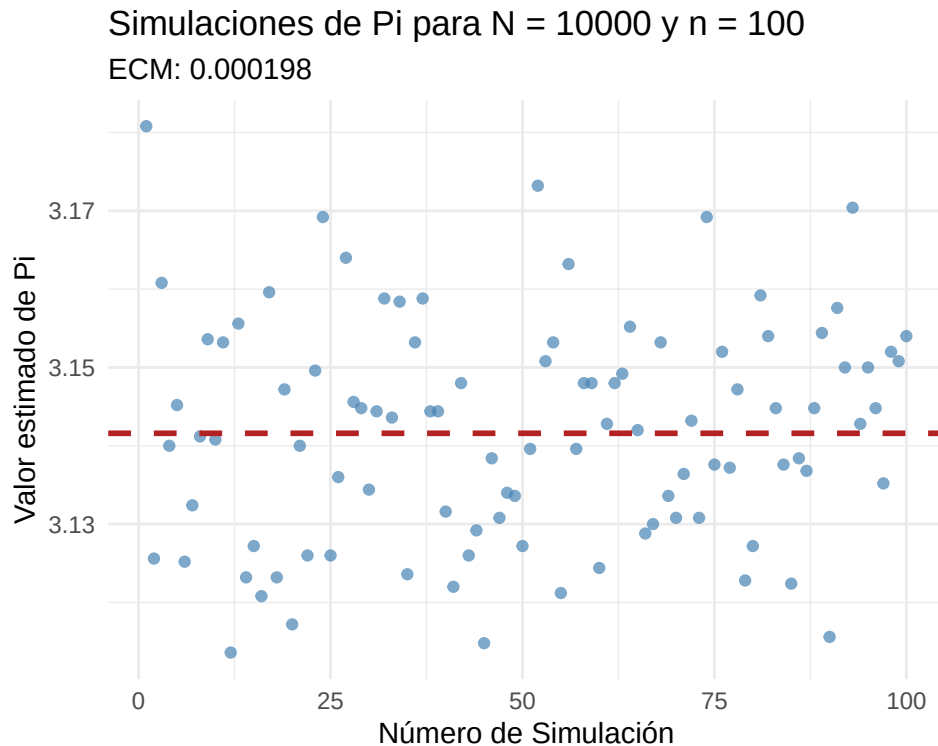


Figure 4: Figura 4: Simulación y cálculo del ECM con un tamaño de muestra de 10000

```
## El valor de pi estimado es: 3.141812
## El calculo del error cuadrático medio (ECM): 0.0001983696

## [1] 0.0001983696
```

Para un n = 100000:

```
## El valor de pi estimado es: 3.142142
## El calculo del error cuadrático medio (ECM): 2.688001e-05

## [1] 2.688001e-05
```

Al comparar las situaciones entre la aguja de Buffon y la estimación a partir de puntos en un cuarto de circunferencia tienen una aproximación con pocos intentos de π , pero se aproxima de manera más rápida el segundo método y es claro que el ECM decrece con mucha rapidez.

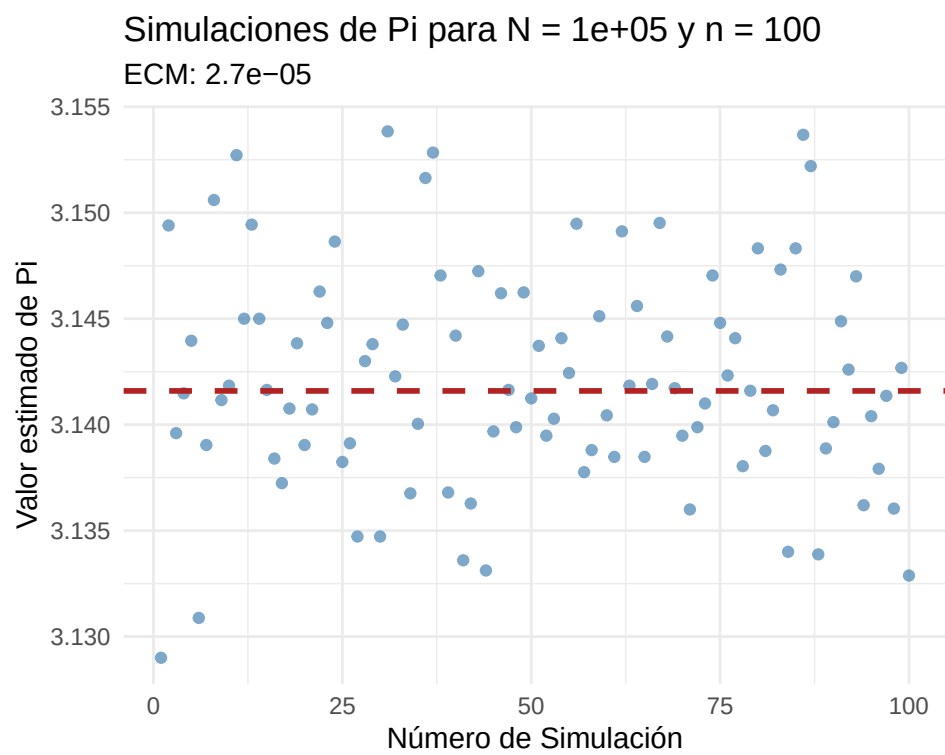


Figure 5: Figura 5: Simulación y cálculo del ECM con un tamaño de muestra de 100000